



KANN AUDITS

Security Review For **Fluton**



Prepared For: **Fluton**

Date Reported: **August 20, 2026**

Audit Summary

Fluton engaged Kann Audits to perform a security review of its confidential DeFi and cross-chain smart contract infrastructure. From June 4 to June 18, a team of security researchers performed a manual review of the in-scope contracts and additionally triaged findings generated by the Kann AI Security Model. All findings identified during the assessment are documented in this report.

Protocol Overview

Fluton is a privacy-focused DeFi protocol that uses fully homomorphic encryption (FHE) to keep token balances and transaction amounts confidential while supporting cross-chain transfers and lending operations. The reviewed system includes confidential ERC-20 wrapping and unwrapping, Zama and CoFHE-based bridge flows for encrypted transfer intents, and a Diamond-based Aave V3 adapter that batches encrypted supply, withdrawal, borrow, and repay requests before asynchronous decryption and settlement. User collateral, debt, and borrowing limits are maintained in encrypted state, while Aave, LayerZero, and FHE gateway integrations provide lending, cross-chain messaging, and decryption infrastructure.

Security Researchers

Ricee	Cosmin	Atiq Ishark
SarveshLimaye	Razkky	stoyanpenchev
VCeb	wadood	parvanj
Digital Forensic	anchabadze	yusufthebdev
Stephen	derastephh	syedghufranhassan
Yunohu	skipper	0xGutzzz
Shreyash	AlexScherbatyuk	Ibukun
Fishy	Luc1jan	jayjoshix
VictoryGod	Shieldrey	Rare_one
Volodymyr	Sachet	

Scope

Files:

- `ZamaBridge.sol`
- `Faucet.sol`
- `cERC20.sol`
- `zama/ConfidentialERC20Wrapped.sol`
- `CoFHEBridge.sol`
- `confidentiality-adapters/aave/Diamond.sol`
- `confidentiality-adapters/aave/libraries/LibAdapterStorage.sol`
- `confidentiality-adapters/aave/libraries/LibDiamond.sol`
- `confidentiality-adapters/aave/libraries/LibSupplyRequest.sol`
- `confidentiality-adapters/aave/libraries/LibWithdrawRequest.sol`
- `confidentiality-adapters/aave/libraries/LibBorrowRequest.sol`
- `confidentiality-adapters/aave/libraries/LibRepayRequest.sol`
- `confidentiality-adapters/aave/facets/AdminFacet.sol`
- `confidentiality-adapters/aave/facets/SupplyFacet.sol`
- `confidentiality-adapters/aave/facets/WithdrawFacet.sol`
- `confidentiality-adapters/aave/facets/BorrowFacet.sol`
- `confidentiality-adapters/aave/facets/RepayFacet.sol`
- `confidentiality-adapters/aave/facets/GetterFacet.sol`
- `confidentiality-adapters/aave/facets/DiamondLoupeFacet.sol`

Methodology

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms, encrypted-state handling, asynchronous gateway callbacks, cross-chain messaging, and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

The auditing process specifically considered:

- Testing against common and uncommon attack vectors
- FHE permission and confidentiality boundary review
- Asynchronous decryption and callback lifecycle analysis
- Aave accounting, collateral, debt, and liquidity-index consistency
- Access control and Diamond facet authorization verification
- Cross-chain message and bridge intent state-transition review
- Manual line-by-line review by experienced security researchers
- Verifying contract logic matches the intended protocol behavior

Risk Classification

- **Critical** – directly exploitable vulnerabilities that can cause catastrophic loss of funds, complete confidentiality failure, or protocol-wide compromise and require immediate remediation.
- **High** – serious vulnerabilities that can cause major loss of funds, broken accounting, protocol-wide disruption, or significant security impact.
- **Medium** – vulnerabilities with meaningful security or availability impact that generally require specific conditions, configurations, or execution paths.
- **Low** – lower-impact security, reliability, accounting, or compatibility issues that do not normally create an immediate protocol-wide exploit by themselves.
- **Informational** – non-exploitable findings, best-practice deviations, usability concerns, or observations that may improve correctness and maintainability.

Findings Count

Critical/High	Medium	Low/Info
12	5	5

Findings Summary

ID	Title	Severity
C-01	Any user can decrypt every other depositor's confidential supply amount for a given asset by triggering batch processing	Critical
C-02	AdminFacet - All External Functions Lack Access Control	Critical
C-03	callbackWithdrawRequest Wrong Decimal Scaling in approve and wrap	Critical
C-04	_setMaxBorrowables - Single-Asset Scaled Balance Overwrites Borrow Limits for All Assets	Critical
H-01	onUnwrap - Enum Default Value (0) Matches RequestType.SUPPLY, Causing Ghost Linkages	High
H-02	Supply/Withdraw Unit Mismatch - Aave Liquidity Index Scaling Never Reversed on Withdrawal, Causing Permanent Principal Loss	High
H-03	Unlimited Re-Borrowing Due to Missing Debt Subtraction in maxBorrowable Check	High
H-04	Withdraw Flow Uses Unnormalized Scaled Balances When Clamping User Withdrawals	High
H-05	Incorrect Cross-Token Subtraction of aToken and Debt Token Scaled Balances in Withdraw Logic	High
H-06	Incorrect Subtraction Between Confidential Token Amount and Scaled Balance Causes Accounting Corruption in Withdraw Flow	High
H-07	Missing Liquidation Handling Causes LTV Desync, Borrow Denial and Broken State Consistency	High
H-08	Gateway Callback Functions Are Missing onlyGateway Access Control	High

ID	Title	Severity
M-01	Unsafe IERC20.approve() Pattern Breaks Compatibility with USDT-Like Tokens	Medium
M-02	Uninitialized matchedIndexes Slots Can DoS Mixed-Asset Supply Batches	Medium
M-03	Decimal Underflow When cToken Decimals Exceed Underlying Asset Decimals	Medium
M-04	Lazy maxBorrowable Caching Uses Stale LTV After Aave Risk Parameters Change	Medium
M-05	Borrow Finalization Rounds User Debt Down Below The Diamond's Actual Aave Liability	Medium
L-01	onUnwrap - Unbounded Backward Loop Scales as O(requestId), Leading to Out-of-Gas	Low
L-02	getSuppliedBalance Returns Scaled Balance Instead of Interest-Accrued Balance	Low
L-03	Encrypted Borrowed Balance Returned Without ACL Permission Prevents Users From Decrypting Debt Value	Low
I-01	Division by Zero in Multiplier Calculation When Total Batch Amount Is Less Than 1e6	Info
I-02	Expired Gateway Deadlines Can Lock Batched Supply And Repay Funds Inside The Adapter	Info

Detailed Findings

C-01 - Any user can decrypt every other depositor's confidential supply amount for a given asset by triggering batch processing

Critical

Summary

When a `supplyRequest()` call causes the global pending-request queue to reach `REQUEST_THRESHOLD`, the function grants the triggering caller (`msg.sender`) FHE decryption rights over the encrypted amount of every matched request for that asset in the batch – not just their own. This breaks the core confidentiality guarantee of the protocol: any user can learn the exact deposit amount of every other user batched alongside them.

Description

Inside `supplyRequest()`, once the pending-request count reaches the threshold, the function loops over all pending requests, collects the ones matching the current asset, and grants decryption access on each one's encrypted amount:

```
for (uint256 i = 0; i < s.supplyRequests.length; i++) {
```

```
    LibAdapterStorage.SupplyRequestData memory srd = s.supplyRequests[i]; if  
(srd.asset == asset) { requests[count] = srd; matchedIndexes[count] = i;
```

```
    TFHE.allow(requests[count].amount, msg.sender); // ← grants msg.sender ac-  
    cess to every matched user's amount TFHE.allowThis(requests[count].amount);
```

```
    unchecked { count++; } if (count == s.REQUEST_THRESHOLD) break; }
```

```
} requests[count] here is another depositor's SupplyRequestData whenever  
srd.sender != msg.sender. The TFHE.allow(requests[count].amount, msg.sender)  
call is unconditional – it does not check whether the matched request actually  
belongs to the caller. As a result, the user whose supplyRequest() call happens  
to push the queue over the threshold is granted permission to request off-chain  
decryption of every other batched user's amount for that asset, via the standard  
fhevm reencrypt/decrypt flow. This is a confidentiality break, not just a logic bug:  
the entire design rationale for routing deposits through FHE – hiding individual  
users' supplied amounts while only revealing the aggregate batch total to Aave –  
is defeated by this single misplaced allow call.
```

Impact

Any user can learn the exact confidential supply amount of any other user, for any asset, with no special timing precision required. Exploitation does not require landing in the exact "last slot" of the threshold – because the threshold check operates on the global pending -request count (`s.supplyRequests.length`) across all assets rather than per -asset, and the fixed -size `requests/matchedIndexes` arrays don't actually verify `count == REQUEST_THRESHOLD` before processing (see separate finding), an attacker only needs their own `supplyRequest()` call to be the one that pushes the global total over threshold while at least one other pending request exists for their chosen asset. This is a substantially wider and easier -to -hit window than precise last -slot timing. No special privileges, cost, or front -running sophistication is required – a single normal user -facing call is sufficient. This fully and trivially defeats the protocol's confidentiality value proposition for batched deposits.

Proof of Concept

User A calls `supplyRequest(assetX, encryptedAmountA, ...)`. Their `cTokens` are pulled and the request is queued. Global pending count is now one below `REQUEST_THRESHOLD`. User B (attacker) calls `supplyRequest(assetX, encryptedAmountB, ...)`, pushing the global count to `REQUEST_THRESHOLD` and triggering the batching branch. Inside the loop, both User A's and User B's requests for `assetX` are matched. `TFHE.allow(requests[i].amount, msg.sender)` runs for both – granting User B (`msg.sender`) decryption rights over User A's encrypted amount. User B requests reencryption/decryption of User A's amount handle off -chain (standard `fhevm` user -decrypt flow) and recovers User A's exact deposit amount.

Recommended Mitigation

Grant decryption access to the original depositor, not the caller who happens to trigger batch processing:

```
TFHE.allow(requests[count].amount, srd.sender);
```

`TFHE.allowThis(requests[count].amount);` `allowThis` should remain as -is – the contract itself still needs continued access to the handle for the subsequent homomorphic summation in `\{}_processSupplyRequests`. Only the per -user `allow` grant needs to target `srd.sender` instead of `msg.sender`.

Remediation

FIXED

C-02 - AdminFacet - All External Functions Lack Access Control

Critical

Location

- Contract: AdminFacet
- File: contracts/confidentiality-adapters/aave/facets/AdminFacet.sol
- Functions: setCTokenAddress() (L17), setAavePoolAddress() (L27), setRequestThreshold() (L33), initMappings() (L38)

Root Cause

The contract defines an `onlyOwner` modifier at line 11 that correctly reads the Diamond owner from `LibDiamond.diamondStorage().contractOwner`. However, none of the four external functions apply this modifier. All four function signatures carry only the `external` visibility with no restriction.

Vulnerable Code

```
// AdminFacet.sol -- L11-L14: modifier exists
modifier onlyOwner() {
    require(msg.sender == LibDiamond.diamondStorage().contractOwner, "AdminFacet: Not
        owner");
    _;
}

// AdminFacet.sol -- L17: modifier NOT applied
function setCTokenAddress(address[] memory tokens, address[] memory cTokens) external
{
    // ...
}

// AdminFacet.sol -- L33: modifier NOT applied
function setRequestThreshold(uint8 threshold) external {
    // ...
}
```

Of these four functions, two – `setCTokenAddress` (selector `0x8db96952`) and `setRequestThreshold` (selector `0x9cc6d79b`) – are registered in the live Diamond at `0x38db4c0630524EfE50783dC9Af76E5f1e4a466c5` through the AdminFacet at `0xac6aB0f8AcaAe5F34AF851eB7186051dDd37F4bf`. A call from any externally-owned account that is not the Diamond owner to these selectors via the Diamond proxy is delegated to the AdminFacet and executes without reverting. This was confirmed by sending transactions from a non-owner address on a Sepolia fork – both calls returned with status 1 (success). As a control, a `diamondCut`

call from the same non-owner address correctly reverts, proving that Diamond-level owner enforcement works for `diamondCut` but is absent on the `AdminFacet` surface.

Impact

An arbitrary address can:

1. Call `setRequestThreshold(0)` – sets `REQUEST_THRESHOLD` to zero. Any subsequent call to `supplyRequest`, `borrowRequest`, `withdrawRequest`, or `repayRequest` immediately enters the batch-processing path because any `array.length >= 0` is always `true`. The zero-sized allocation and subsequent division-by-zero in `_processSupplyRequests` causes a permanent revert, halting all four request types.
2. Call `setCTokenAddress(tokens, maliciousAddresses)` – remaps the `cToken` `<->` underlying-token relationship used throughout the protocol. All subsequent `supplyRequest`, `callbackSupplyRequest`, `callbackWithdrawRequest`, and `finalizeSupplyRequests` calls will operate on the attacker-controlled token contract, allowing theft of all funds flowing through the adapter.

Combined, these two operations give an attacker full control of protocol operations without requiring the Diamond owner's private key.

Recommendation

Apply the existing `onlyOwner` modifier to all four external functions:

```
function setCTokenAddress(address[] memory tokens, address[] memory cTokens) external
    onlyOwner { ... }
function setAavePoolAddress(address pool, address poolDataProvider) external
    onlyOwner { ... }
function setRequestThreshold(uint8 threshold) external onlyOwner { ... }
function initMappings(address user, address[] memory tokens) external onlyOwner { ...
}
```

Remediation

FIXED

C-03 - callbackWithdrawRequest Wrong Decimal Scaling in approve and wrap

Critical

Location

- Contract: LibWithdrawRequest (library, executed via Diamond delegatecall)
- File: contracts/confidentiality-adapters/aave/libraries/LibWithdrawRequest.sol
- Function: callbackWithdrawRequest() – lines 119-131

Root Cause

The function correctly computes the Aave withdrawal amount in the underlying token's native decimals:

```
uint256 amountToWithdraw = amount *
    (10 ** (IERC20Metadata(asset).decimals() - ConfidentialERC20Wrapped(cToken).
        decimals()));
```

It then correctly withdraws from Aave:

```
s.aavePool.withdraw(asset, amountToWithdraw, address(this));
```

However, the next two operations re-use the raw amount variable (which is in 6-decimal cToken scale) instead of amountToWithdraw:

Vulnerable Code

```
// LibWithdrawRequest.sol -- L129-L130
IERC20(asset).approve(cToken, amount);           // BUG: should be
    amountToWithdraw
ConfidentialERC20Wrapped(cToken).wrap(amount);    // BUG: should be
    amountToWithdraw
```

Scenario

For an 18-decimal underlying asset (e.g. WETH) with a 6-decimal cToken:

1. Gateway callback delivers amount = 1_000_000 (1 token in 6-decimal scale).
2. amountToWithdraw = 1_000_000 x 10¹² = 1_000_000_000_000_000_000 (1e18) – correct.
3. s.aavePool.withdraw(asset, 1e18, address(this)) – withdraws 1 WETH to the Diamond. Correct.
4. IERC20(asset).approve(cToken, 1_000_000) – approves 1e6 wei of WETH (~0). Wrong.

5. `ConfidentialERC20Wrapped(cToken).wrap(1_000_000)` – the `wrap()` function internally computes `amountAdjusted = 1_000_000 / 1012 = 0` due to integer truncation. Zero `cTokens` are minted.

Result: the Diamond contract now holds 1 WETH (minus the 1e6 wei transferred to `wrap`), but zero `cTokens` were minted for the user. The remaining ~1 WETH is permanently stranded with no recovery mechanism.

This stands in contrast to `callbackSupplyRequest()` in `LibSupplyRequest.sol` (L122-L125), which correctly computes `unwrappedAmount` and passes it to `approve()`.

Impact

Every withdrawal callback for any token with more than 6 decimals results in the underlying tokens being permanently locked in the Diamond. Users receive zero `cTokens` despite the successful Aave withdrawal. The tokens have no recovery path since no admin function or emergency withdrawal mechanism exists for stranded ERC-20s.

Recommendation

Replace the raw `amount` argument with `amountToWithdraw` in both the `approve` and `wrap` calls:

```
IERC20(asset).approve(cToken, amountToWithdraw);
ConfidentialERC20Wrapped(cToken).wrap(amountToWithdraw);
```

Remediation

FIXED

C-04 - _setMaxBorrowables - Single-Asset Scaled Balance Overwrites Borrow Limits for All Assets

Critical

Location

- File: contracts/confidentiality-adapters/aave/libraries/LibSupplyRequest.sol
_setMaxBorrowables(), L183-L198
- File: contracts/confidentiality-adapters/aave/libraries/LibWithdrawRequest.sol
_setMaxBorrowables(), L182-L197
- File: contracts/confidentiality-adapters/aave/libraries/LibBorrowRequest.sol
_setMaxBorrowables(), L222-L237
- File: contracts/confidentiality-adapters/aave/libraries/LibRepayRequest.sol
_setMaxBorrowables() (identical logic)

The function is copy-pasted identically in all four library files.

Root Cause

_setMaxBorrowables(euint64 currentBalance, address sender) receives the scaledBalance of a single asset and loops over all entries in aaveAssets[], unconditionally overwriting userMaxBorrowablePerAsset[sender][asset] for every asset using only that one balance:

Vulnerable Code

```
// LibSupplyRequest.sol -- L183-L198
function _setMaxBorrowables(euint64 currentBalance, address sender) internal {
    LibAdapterStorage.Storage storage s = LibAdapterStorage.getStorage();
    address[] memory aaveAssets = s.aaveAssets;
    for (uint256 i = 0; i < aaveAssets.length; i++) {
        address asset = aaveAssets[i];
        (, uint256 ltv, , , , , , , ) = s.aaveDataProvider.
            getReserveConfigurationData(asset);
        s.userMaxBorrowablePerAsset[sender][asset] = TFHE.div(
            TFHE.mul(currentBalance, uint64(ltv)), uint64(10000)
        );
        // ...
    }
}
```

This function is called from _processStateUpdates in each library, passing the most recently updated scaledBalances[sender][asset] for the current finalized asset only. Each invocation unconditionally replaces all values previously stored for all other assets.

Scenario

1. User supplies 1000 USDC. Finalization calls `_setMaxBorrowables(1_000_000_000, user)`. At 82.5% LTV: `maxBorrowable[USDC] = 825_000_000`, `maxBorrowable[WETH] = 800_000_000`, etc. Correct at this point.
2. User supplies 2 WETH. Finalization calls `_setMaxBorrowables(2_000_000, user)`. Now: `maxBorrowable[USDC] = 1_650_000`, `maxBorrowable[WETH] = 1_600_000`. The USDC `maxBorrowable` dropped by 99.8% – the 1000 USDC of collateral is completely forgotten.
3. Conversely, if a user supplies a small amount of asset A and then a large amount of asset B, all `maxBorrowable` entries are inflated to reflect only asset B's balance. This can inflate borrow limits by 400x or more for small positions.

Impact

The cross-collateral model is fundamentally broken. Each finalization for any asset erases the collateral contribution of all other assets. Depending on the order of finalization, users either have their borrow capacity reduced to near-zero (denial of service) or inflated far beyond their actual collateral value (potential under-collateralized borrowing). Since the adapter holds an aggregate Aave position for all users, an under-collateralized borrow state can trigger a liquidation of the entire pooled position.

Recommendation

Redesign `_setMaxBorrowables` to aggregate across all collateral positions for the user. Instead of accepting a single `currentBalance`, the function should iterate `aaveAssets` and read `scaledBalances[sender][asset_i]` for each, computing the per-asset max borrowable from its own collateral balance:

```
function _setMaxBorrowables(address sender) internal {
    LibAdapterStorage.Storage storage s = LibAdapterStorage.getStorage();
    address[] memory aaveAssets = s.aaveAssets;
    for (uint256 i = 0; i < aaveAssets.length; i++) {
        address asset = aaveAssets[i];
        uint64 balance = s.scaledBalances[sender][asset];
        (, uint256 ltv, , , , , , , ) = s.aaveDataProvider.
            getReserveConfigurationData(asset);
        s.userMaxBorrowablePerAsset[sender][asset] = TFHE.div(
            TFHE.mul(balance, uint64(ltv)), uint64(10000)
        );
        // ...
    }
}
```

Remediation

FIXED

H-01 - onUnwrap - Enum Default Value (0) Matches RequestType.SUPPLY, Causing Ghost Linkages

High

Location

- Contract: Diamond
- File: contracts/confidentiality-adapters/aave/Diamond.sol – onUnwrap(), L68-L82
- File: contracts/confidentiality-adapters/aave/libraries/LibAdapterStorage.sol – RequestType enum, L12-L17

Root Cause

The RequestType enum defines SUPPLY = 0 as its first member:

```
// LibAdapterStorage.sol -- L12-L17
enum RequestType {
    SUPPLY,      // = 0
    WITHDRAW,    // = 1
    BORROW,      // = 2
    REPAY        // = 3
}
```

In onUnwrap(), the backward loop reads requestIdToRequestData[i].requestType and compares it against SUPPLY or REPAY:

Vulnerable Code

```
// Diamond.sol -- L73-L78
for (uint256 i = requestId - 1; i > 0; i--) {
    LibAdapterStorage.RequestType rt = s.requestIdToRequestData[i].requestType;
    if (rt == LibAdapterStorage.RequestType.SUPPLY || rt == LibAdapterStorage.
        RequestType.REPAY) {
        s.requestIdToUnwrapRequestId[i] = requestId;
        break;
    }
}
```

In Solidity, reading a mapping key that has never been written returns the default value for the type. For a struct containing a RequestType enum, the default is 0, which is RequestType.SUPPLY. Therefore, any requestIdToRequestData[i] where i was never assigned – including IDs for deleted requests and IDs that never corresponded to any request – will return RequestType.SUPPLY and satisfy the condition.

Impact

The loop will almost always match the very first `requestId - 1` entry that it reads, regardless of whether a real SUPPLY or REPAY request ever existed at that ID. This links the unwrap amount to the wrong request ID. Finalization of the actual SUPPLY/REPAY request will then either read a stale or incorrect unwrap amount, or find no linked unwrap at all, causing an `AmountIsZero` revert.

Recommendation

Add a `NONE = 0` sentinel value as the first enum member so that uninitialized storage does not collide with valid request types:

```
enum RequestType {
    NONE,          // = 0, sentinel for uninitialized
    SUPPLY,        // = 1
    WITHDRAW,      // = 2
    BORROW,        // = 3
    REPAY          // = 4
}
```

Update the `onUnwrap` check to exclude `NONE`.

Remediation

FIXED

H-02 - Supply/Withdraw Unit Mismatch - Aave Liquidity Index Scaling Never Reversed on Withdrawal, Causing Permanent Principal Loss

High

Summary

The Diamond's supply finalization converts user deposits from cToken units into Aave-scaled units using a `multiplier` derived from the Aave liquidity index. The resulting `scaledBalance` is stored as the user's position. However, the withdrawal path converts `scaledBalance` back to an ERC20 amount using only a decimal difference adjustment, never reversing the Aave liquidity index scaling. This causes the withdrawal amount passed to `aavePool.withdraw()` to be denominated in Aave-scaled units rather than underlying units – resulting in the user receiving less than they deposited.

On any Aave market where the liquidity index exceeds 1.0 (which is every active market on mainnet), every depositor through the Diamond suffers immediate, permanent principal loss proportional to $1 - (1 / \text{liquidityIndex})$. At mainnet USDC's current liquidity index of ~1.06, this equates to a 5.66% loss per deposit.

Vulnerability Detail

Supply Path: Deposits Are Scaled Down

In `LibSupplyRequest.finalizeSupplyRequests` (`contracts/confidentiality-adapters/aave/libraries/LibSupplyRequest.sol#L141-L148`), the Diamond supplies the decrypted batch total to Aave and computes a `multiplier` from the Aave scaled balance delta:

```
uint256 beforeScaledBalance = IScaledBalanceToken(aToken).scaledBalanceOf(address(this));
s.aavePool.supply(asset, amount, address(this), requests[0].referralCode);
uint256 afterScaledBalance = IScaledBalanceToken(aToken).scaledBalanceOf(address(this));
uint256 difference = afterScaledBalance - beforeScaledBalance;

uint256 multiplier = difference / (amount / (10 ** 6)); // <- incorporates liquidityIndex
```

In Aave V3, `scaledBalanceOf` returns `balance / liquidityIndex`. So when `liquidityIndex > 1.0`, the difference is smaller than `amount`, producing a `multiplier < 106`.

Each user's `scaledBalance` is then updated in `_processStateUpdates`

(contracts/confidentiality-adapters/aave/libraries/LibSupplyRequest.sol#L172-L176):

```
uint64 newBalance = TFHE.add(
    s.scaledBalances[sender][asset],
    TFHE.div(TFHE.mul(amount, uint64(multiplier)), 1e6) // <- produces value LESS
    than deposit
);
```

Example at liquidityIndex = 1.05:

- User deposits 1000 USDC (= 109 in 6-decimal)
- Aave scaled delta = $109 / 1.05 = 952,380,952$
- multiplier = $952,380,952 / (109 / 106) = 952,380$
- scaledBalance = $109 \times 952,380 / 106 = 952,380,000$

The user deposited 109 but their scaledBalance records only 952,380,000 – a 4.76% reduction.

Withdraw Path: Scaling Is Never Reversed

In LibWithdrawRequest.callbackWithdrawRequest (contracts/confidentiality-adapters/aave/libraries/LibWithdrawRequest.sol#L119-L122), the decrypted withdrawal amount (which is capped by the user's scaledBalance) is converted to an ERC20 amount using only the decimal difference:

```
uint256 amountToWithdraw = amount * (10 ** (IERC20Metadata(asset).decimals() - cToken
    .decimals()));
s.aavePool.withdraw(asset, amountToWithdraw, address(this));
```

For USDC (6/6 decimals), decimals_diff = 0, so amountToWithdraw = amount – the raw scaledBalance value is passed directly to Aave's withdraw().

Aave's pool.withdraw(asset, amount, to) interprets the amount parameter as an underlying token amount (not a scaled amount). It returns exactly amount underlying tokens and burns amount / currentLiquidityIndex scaled aTokens.

Continuing the example:

- User requests withdrawal of their full scaledBalance = 952,380,000
- amountToWithdraw = $952,380,000 \times 100 = 952,380,000$
- aavePool.withdraw(USDC, 952,380,000, Diamond) -> returns 952.38 USDC

The user deposited 1000 USDC but receives only 952.38 USDC. 47.62 USDC of principal is permanently lost.

The User Cannot Withdraw Their Full Deposit

Even if the user requests their original deposit amount (109), the `withdrawRequest` (contracts/confidentiality-adapters/aave/libraries/LibWithdrawRequest.sol#L21-L25) check prevents it:

```
uint64 withdrawableScaledBalance = TFHE.sub(suppliedScaledBalance, debtScaledBalance);
uint64 safeAmount = TFHE.select(
    TFHE.le(amount, withdrawableScaledBalance), // 10^9 <= 952,380,000? -> FALSE
    amount,
    TFHE.asEuint64(0) // -> safeAmount = 0
);
```

The user's `scaledBalance` (952,380,000) is less than their deposit (109), so requesting the full deposit amount results in `safeAmount = 0` – the withdrawal is silently zeroed out.

The Missing Conversion

The correct withdrawal conversion should multiply the scaled balance by the current liquidity index to obtain the actual underlying value:

```
actualUnderlying = scaledBalance x currentLiquidityIndex / RAY
```

But the code only applies a decimal adjustment:

```
amountToWithdraw = scaledBalance x 10^(decimals_diff)
```

This means the Aave scaling applied during supply is never reversed during withdrawal.

Impact Scales With Market Maturity The principal loss percentage equals $1 - (1 / \text{liquidityIndex})$:

Aave Market	Approx. liquidityIndex	Principal Loss Per Deposit
-----	-----	-----
Fresh pool (no borrows yet)	1.000	0%
WETH (mainnet)	~1.03	~2.9%
USDT (mainnet)	~1.05	~4.8%
USDC (mainnet)	~1.06	~5.7%**
Mature market (years)	~1.20	~16.7%**
Extreme (very old market)	~2.00	~50%**

The liquidity index only increases over time. Every deposit through the Diamond on any production Aave market suffers this loss.

Impact

Every user who deposits through the Diamond on any Aave market with `liquidityIndex > 1.0` permanently loses a percentage of their principal. On

mainnet Aave USDC (the most common asset), this is approximately 5.66% per deposit at current market conditions. The loss is:

- Immediate: Occurs at the moment of supply finalization, not over time.
- Permanent: No mechanism exists to reclaim the lost principal.
- Invisible: The user cannot see their encrypted `scaledBalance` to detect the shortfall.
- Compounding: Repeat deposits compound the loss (each deposit loses another ~5.66%).

Additionally, Aave yield accrual is never reflected in users' `scaledBalance` values. Even the earliest depositors (who avoid principal loss because `liquidityIndex` ~ 1.0 at pool inception) never receive interest – yield accumulates in the Diamond's aggregate Aave position permanently. There is no distribution, sweep, harvest, or claim function anywhere in the codebase.

Code Snippet

Supply multiplier calculation – incorporates `liquidityIndex`, shrinking `scaledBalance`:

<https://github.com/example/contracts/confidentiality-adapters/aave/libraries/LibSupplyRequest.sol#L141-L176>

```
uint256 beforeScaledBalance = IScaledBalanceToken(aToken).scaledBalanceOf(address(
    this));
s.aavePool.supply(asset, amount, address(this), requests[0].referralCode);
uint256 afterScaledBalance = IScaledBalanceToken(aToken).scaledBalanceOf(address(this
));
uint256 difference = afterScaledBalance - beforeScaledBalance;
uint256 multiplier = difference / (amount / (10 ** 6));

// User's balance update:
uint64 newBalance = TFHE.add(
    s.scaledBalances[sender][asset],
    TFHE.div(TFHE.mul(amount, uint64(multiplier)), 1e6) // scaledBalance <
    depositAmount
);
```

Withdrawal callback – only applies decimal adjustment, never reverses Aave scaling:

<https://github.com/example/contracts/confidentiality-adapters/aave/libraries/LibWithdrawRequest.sol#L119-L122>

```
uint256 amountToWithdraw = amount *
    (10 ** (IERC20Metadata(asset).decimals() - ConfidentialERC20Wrapped(cToken).
    decimals()));
s.aavePool.withdraw(asset, amountToWithdraw, address(this));
// amountToWithdraw is in Aave-scaled units, but Aave treats it as underlying ->
    principal loss
```

Recommendation

The withdrawal callback must reverse the Aave scaling by multiplying the decrypted amount by the current liquidity index before passing it to `aavePool.withdraw()`:

```
function callbackWithdrawRequest(uint256 requestId, uint64 amount) internal {
    // ...
-    uint256 amountToWithdraw = amount *
-    (10 ** (IERC20Metadata(asset).decimals() - ConfidentialERC20Wrapped(cToken)
    .decimals())));
+    // Reverse the Aave scaling: convert from scaled units back to underlying
+    uint256 liquidityIndex = s.aavePool.getReserveNormalizedIncome(asset);
+    uint256 scaledAsUnderlying = (uint256(amount) * liquidityIndex) / 1e27;
+    uint256 amountToWithdraw = scaledAsUnderlying *
+    (10 ** (IERC20Metadata(asset).decimals() - ConfidentialERC20Wrapped(cToken)
    .decimals())));

    s.aavePool.withdraw(asset, amountToWithdraw, address(this));

-    IERC20(asset).approve(cToken, amount);
-    ConfidentialERC20Wrapped(cToken).wrap(amount);
+    // Wrap the actual withdrawn amount (in cToken-scale)
+    uint256 cTokenAmount = amountToWithdraw /
+    (10 ** (IERC20Metadata(asset).decimals() - ConfidentialERC20Wrapped(cToken)
    .decimals())));
+    IERC20(asset).approve(cToken, cTokenAmount);
+    ConfidentialERC20Wrapped(cToken).wrap(cTokenAmount);
    // ...
}
```

The same fix should be applied to the borrow and repay callbacks that perform similar decimal-only conversions.

For yield distribution, the protocol should implement one of:

1. Periodic rebase: A keeper function that reads `aavePool.getReserveNormalizedIncome(asset)` and proportionally updates all users' `scaledBalances` via FHE operations.
2. Withdrawal-time yield capture: Compute the actual underlying value at withdrawal time (as shown in the fix above) so users automatically receive their share of accrued interest.
3. Admin sweep: At minimum, add an owner-only function to extract trapped yield from the Diamond's Aave position for manual distribution.

Option 2 (the fix above) is the simplest and most correct approach – it ensures each user's withdrawal reflects the current liquidity index, naturally including any accrued yield.

Remediation

FIXED

H-03 - Unlimited Re-Borrowing Due to Missing Debt Subtraction in maxBorrowable Check

High

Summary

The borrow cap check in `LibBorrowRequest.borrowRequest()` validates that the requested borrow amount does not exceed `maxBorrowable`, which is computed as `scaledBalance * ltv / 10000`. This formula represents the user's total borrowing capacity but never subtracts the user's existing `scaledDebts`. As a result, `maxBorrowable` remains constant regardless of how much debt the user has already accumulated, allowing unlimited repeat borrowing against the same collateral. An attacker can drain the Diamond's entire Aave borrowing capacity, stealing other users' collateral through the pooled aggregate position.

Vulnerability Detail

The Borrow Cap Check

When a user submits a borrow request, the only guard is at `LibBorrowRequest.sol`##L26-27 (`contracts/confidentiality-adapters/aave/libraries/LibBorrowRequest.sol`#L26-L27):

```
uint64 maxBorrowable = s.userMaxBorrowablePerAsset[msg.sender][asset];
uint64 safeAmount = TFHE.select(TFHE.le(amount, maxBorrowable), amount, TFHE.asEuint64(0));
```

If `amount <= maxBorrowable`, the full requested amount is queued. Otherwise, 0 is silently queued (the FHE select clamps it).

How maxBorrowable Is Computed

`maxBorrowable` is set by `_setMaxBorrowables()`, which is called after every supply, borrow, withdraw, and repay finalization. The implementation is identical across all four libraries (`LibBorrowRequest.sol`##L220-234 (`contracts/confidentiality-adapters/aave/libraries/LibBorrowRequest.sol`#L220-L234)):

```
function _setMaxBorrowables(uint64 currentBalance, address sender) internal {
    LibAdapterStorage.Storage storage s = LibAdapterStorage.getStorage();
    address[] memory aaveAssets = s.aaveAssets;
    for (uint256 i = 0; i < aaveAssets.length; i++) {
        address asset = aaveAssets[i];
        (, uint256 ltv, , , , , , , ) = s.aaveDataProvider.getReserveConfigurationData(asset);
        s.userMaxBorrowablePerAsset[sender][asset] = TFHE.div(
            TFHE.mul(currentBalance, uint64(ltv)), uint64(10000)
        );
    }
}
```



```
}  
}
```

The formula is:

```
maxBorrowable = scaledBalance x ltv / 10000
```

There is no subtraction of `scaledDebts[user][asset]` anywhere. The function does not read, reference, or account for existing user debt.

Why `maxBorrowable` Never Decreases After Borrowing

After a borrow finalizes, the state update in `_processStateUpdates` (contracts/confidentiality-adapters/aave/libraries/LibBorrowRequest.sol#L191-L218) is:

```
// L202-206: Debt increases  
s.scaledDebts[user][asset] = TFHE.add(s.scaledDebts[user][asset],  
    borrowedScaledAmount);  
  
// L211: Read the SUPPLY balance (not modified by borrowing)  
euint64 scaledBalance = s.scaledBalances[user][asset];  
  
// L213: Recalculate maxBorrowable from unchanged supply balance  
_setMaxBorrowables(scaledBalance, user);
```

Borrowing increases `scaledDebts` but does not modify `scaledBalances`. Since `_setMaxBorrowables` only reads `scaledBalances` (via the `currentBalance` parameter), it recomputes the exact same `maxBorrowable` value as before the borrow. The newly increased debt is invisible to the cap.

Step-by-Step Attack

Assume a single asset (USDC) with 75% LTV, `REQUEST_THRESHOLD = 3`, and the Diamond has a healthy aggregate Aave position from other users' deposits.

Setup: Attacker supplies 1000 cUSDC to the Diamond.

After supply finalization:

- `scaledBalances[attacker][USDC] = enc(1000)`
- `scaledDebts[attacker][USDC] = enc(0)`
- `maxBorrowable[attacker][USDC] = enc(750) (1000 x 75%)`

Borrow #1: Attacker submits `borrowRequest(USDC, enc(750))`.

- Check: `TFHE.le(enc(750), enc(750)) -> enc(true) -> safeAmount = enc(750)` [OK]
- Batch accumulates to threshold, processes, decrypts sum, finalizes.

- Post-finalize state:
- `scaledDebts[attacker][USDC] = enc(~750)`
- `scaledBalances[attacker][USDC] = enc(1000) <- unchanged`
- `_setMaxBorrowables(enc(1000), attacker) -> maxBorrowable = enc(750) <- unchanged`
- Attacker receives 750 cUSDC from Diamond.

Borrow #2: Attacker submits `borrowRequest(USDC, enc(750))` again.

- Check: `TFHE.le(enc(750), enc(750)) -> enc(true) -> safeAmount = enc(750)` [OK] passes again
- `scaledDebts` (currently `enc(750)`) is never consulted in the check.
- Batch processes, finalizes. Attacker receives another 750 cUSDC.
- Post-finalize:
- `scaledDebts[attacker][USDC] = enc(~1500)`
- `maxBorrowable[attacker][USDC] = enc(750) <- still unchanged`

Borrow #N: The same check passes every time. The attacker can repeat indefinitely as long as the Diamond's aggregate Aave position (backed by other users' collateral) remains healthy enough for Aave to allow the borrow.

Withdrawal: After accumulating sufficient debt, the attacker can also withdraw their original supply:

- Withdraw check: `withdrawable = TFHE.sub(scaledBalance=enc(1000), scaledDebt=enc(1500))`
- In Zama's TFHE, `euint64` subtraction wraps modularly: `1000 - 1500 -> 2^64 - 500` (a huge value)
- `TFHE.le(enc(1000), enc(2^64-500)) -> enc(true) -> withdrawal passes`
- Attacker retrieves their 1000 cUSDC supply on top of the borrowed amounts.

Final tally: Attacker deposited 1000, extracted 1000 (withdrawal) + Nx750 (borrows). Net theft = Nx750, sourced from other users' share of the Diamond's pooled Aave position.

Why Aave Itself Doesn't Prevent This

The Diamond contract is a single Aave user. Aave sees one aggregated position:

- Total collateral = sum of all users' supplies

- Total debt = sum of all users' borrows

As long as the aggregate health factor ($\text{totalCollateral} \times \text{avgLTV} / \text{totalDebt} > 1$), Aave allows new borrows. With sufficient collateral from innocent users, the attacker's repeated borrows don't push the aggregate health factor below 1 until the damage is already catastrophic.

The per-user accounting inside the Diamond (encrypted $\text{scaledBalances} / \text{scaledDebts} / \text{maxBorrowable}$) is the only mechanism protecting individual users from each other. This mechanism is broken.

Impact

An attacker can borrow an unlimited amount against a fixed collateral position by repeatedly calling `borrowRequest()`. Each borrow passes the cap check because `maxBorrowable` never decreases – the formula $\text{scaledBalance} \times \text{ltv} / 10000$ ignores existing debt entirely.

The borrowed funds are sourced from the Diamond's aggregate Aave position, which is backed by all users' collateral. This constitutes direct theft from other depositors. Additionally, the `TFHE.sub` underflow in the withdraw path (when $\text{scaledDebts} > \text{scaledBalances}$) wraps to a huge value, allowing the attacker to also withdraw their original supply.

The attack requires only:

1. A supply position in the Diamond.
2. Enough other users' deposits to keep the Diamond's aggregate Aave health factor > 1 .
3. Patience to wait for `REQUEST_THRESHOLD` borrow requests to accumulate per batch (the attacker can use multiple addresses or wait for organic traffic).

Code Snippet

The borrow cap check – no debt subtraction:

<https://github.com/example/contracts/confidentiality-adapters/aave/libraries/LibBorrowRequest.sol#L25-L27>

```
// check if user has enough supply
uint64 maxBorrowable = s.userMaxBorrowablePerAsset[msg.sender][asset];
uint64 safeAmount = TFHE.select(TFHE.le(amount, maxBorrowable), amount, TFHE.
    asEuint64(0));
```

The `_setMaxBorrowables` function – debt never subtracted:

<https://github.com/example/contracts/confidentiality-adapters/aave/libraries/LibBorrowRequest.sol#L220-L234>

```
function _setMaxBorrowables(euint64 currentBalance, address sender) internal {
    // ...
    s.userMaxBorrowablePerAsset[sender][asset] = TFHE.div(
        TFHE.mul(currentBalance, uint64(ltv)), uint64(10000)
    );
    // scaledDebts[sender][asset] is never referenced
}
```

Post-borrow state update – scaledBalances unchanged, maxBorrowable unchanged:

<https://github.com/example/contracts/confidentiality-adapters/aave/libraries/LibBorrowRequest.sol#L211-L213>

```
euint64 scaledBalance = s.scaledBalances[user][asset]; // not modified by borrow
_setMaxBorrowables(scaledBalance, user);                // recomputes same value
```

Recommendation

Subtract existing encrypted debt from maxBorrowable in _setMaxBorrowables, and use the net remaining capacity in the borrow check:

```
function _setMaxBorrowables(euint64 currentBalance, address sender) internal {
    LibAdapterStorage.Storage storage s = LibAdapterStorage.getStorage();
    address[] memory aaveAssets = s.aaveAssets;
    for (uint256 i = 0; i < aaveAssets.length; i++) {
        address asset = aaveAssets[i];
        (, uint256 ltv, , , , , , ) = s.aaveDataProvider.
            getReserveConfigurationData(asset);
-        s.userMaxBorrowablePerAsset[sender][asset] = TFHE.div(
-            TFHE.mul(currentBalance, uint64(ltv)), uint64(10000)
-        );
+        euint64 totalCapacity = TFHE.div(
+            TFHE.mul(currentBalance, uint64(ltv)), uint64(10000)
+        );
+        euint64 existingDebt = s.scaledDebts[sender][asset];
+        ebool hasCapacity = TFHE.gt(totalCapacity, existingDebt);
+        s.userMaxBorrowablePerAsset[sender][asset] = TFHE.select(
+            hasCapacity,
+            TFHE.sub(totalCapacity, existingDebt),
+            TFHE.asEuint64(0)
+        );
    }
}
```

This ensures maxBorrowable represents the remaining borrowing capacity after existing debt, and uses TFHE.select to clamp at 0 instead of underflowing when debt exceeds capacity.

Additionally, consider re-validating caps at finalization time (not just at request time) to mitigate the secondary staleness concern where concurrent requests of different types can bypass each other's effects during the async decryption window.

Remediation

FIXED

H-04 - Withdraw Flow Uses Unnormalized Scaled Balances When Clamping User Withdrawals

High

Description

Within the Aave confidentiality adapter, `scaledBalances` and `scaledDebts` are the protocol's internal Aave-index-scaled accounting values, stored in the adapter's 6-decimal confidential token denomination. The codebase itself shows this model on the supply, borrow, and repay paths:

- `LibSupplyRequest.finalizeSupplyRequests()` measures the Diamond's Aave `scaledBalanceOf(...)` delta and converts it back into the adapter's confidential accounting before writing `s.scaledBalances`.
- `LibBorrowRequest.finalizeBorrowRequests()` does the same with Aave variable debt deltas before writing `s.scaledDebts`.
- `LibRepayRequest.repayRequest()` then acknowledges that `s.scaledDebts` is not a real balance by first converting it into `realDebt` with `getReserveNormalizedVariableDebt(asset)` before comparing it to the user's requested repay amount.

The withdraw path does not follow that same unit discipline. In `LibWithdrawRequest.withdrawRequest()`, the protocol computes:

```
euint64 suppliedScaledBalance = s.scaledBalances[msg.sender][asset];
euint64 debtScaledBalance = s.scaledDebts[msg.sender][asset];
euint64 withdrawableScaledBalance = TFHE.sub(suppliedScaledBalance, debtScaledBalance);
euint64 safeAmount = TFHE.select(TFHE.le(amount, withdrawableScaledBalance), amount, TFHE.asEuint64(0));
```

This compares a user-requested withdraw `amount` against an unnormalized subtraction of two scaled values. That is inconsistent with the repay flow, where the protocol first normalizes scaled debt into `realDebt` before doing the clamp:

```
uint256 reserveNormalizedDebt = s.aavePool.getReserveNormalizedVariableDebt(asset);
euint256 realDebt = TFHE.div(TFHE.mul(TFHE.asEuint256(scaledDebt), reserveNormalizedDebt), 1e27);
euint64 safeAmount = TFHE.select(TFHE.le(amount, TFHE.asEuint64(realDebt)), amount, TFHE.asEuint64(realDebt));
```

In protocol context, this matters because the requested withdraw amount is later treated as a real withdraw amount: `callbackWithdrawRequest()` converts it into underlying ERC20 units and calls `aavePool.withdraw(...)`. The stored `scaledBalances` value is therefore being used as if it's already normalised, even

though the code elsewhere treats analogous scaled debt as requiring normalization first.

The same conceptual mismatch appears in `GetterFacet.getSuppliedBalance()`, whose `NatSpec` says it returns the user's "real supplied balance (encrypted) including interest accrual" while the implementation simply returns `s.scaledBalances[user][asset]` without applying `getReserveNormalizedIncome(asset)`.

Impact

The protocol can understate a user's real withdrawable balance and reject or truncate otherwise valid withdraw requests, especially after interest accrual has increased the real collateral value above the stored scaled value.

Root Cause

The withdraw path mixes scaled accounting and real accounting in the same comparison.

Recommendation

Normalize the supply side before clamping withdraw requests, just as the repay flow normalizes debt before clamping repay requests.

Specifically:

- compute `realSupply = scaledBalances * reserveNormalizedIncome / 1e27`
- compute `realDebt = scaledDebts * reserveNormalizedVariableDebt / 1e27`
- compare the requested withdraw amount against `realWithdrawable = min(realSupply - realDebt, amount)` instead of against `scaledBalances - scaledDebts`
- during withdraw finalization, update `s.scaledBalances` using the actual scaled delta from Aave, for example by measuring `scaledBalanceOf(address(this))` before and after the withdraw, or by converting the withdrawn real amount back into scaled units with the current normalized income
- fix `GetterFacet.getSuppliedBalance()` so it either returns the normalized real supplied balance or is renamed/documented clearly as a scaled balance getter

Remediation

FIXED

H-05 - Incorrect Cross-Token Subtraction of aToken and Debt Token Scaled Balances in Withdraw Logic

High

Description

In `withdrawRequest`, the protocol computes a user's withdrawable balance using:

```
withdrawableScaledBalance = suppliedScaledBalance - debtScaledBalance;
```

where `suppliedScaledBalance` corresponds to aToken (collateral) scaled balance and `debtScaledBalance` corresponds to variable debt token scaled balance

This subtraction is fundamentally incorrect because aTokens and debt tokens represent different accounting systems. They are not directly comparable or subtractable quantities. By mixing these two independent token domains, the protocol produces an invalid withdrawable amount that does not reflect real borrowing health or liquidation constraints.

Root Cause

Incorrect assumption that `collateral - debt = withdrawable liquidity` is valid in scaled token space. The contract does not normalize the balances into a common value domain (e.g., collateral token) with accrued interest

Impact

users can withdraw unsafe amounts or be unfairly restricted. Scaled token arithmetic does not represent real economic value

Remediation

FIXED

H-06 - Incorrect Subtraction Between Confidential Token Amount and Scaled Balance Causes Accounting Corruption in Withdraw Flow

High

Description

In `_processStateUpdates` of `LibWithdrawRequest`, the protocol directly subtracts `requests[i].amount` (a confidential token amount) from `scaledBalances`:

```
s.scaledBalances[user][asset] = TFHE.sub(  
    s.scaledBalances[user][asset],  
    requests[i].amount  
);
```

This is fundamentally incorrect because:

- `scaledBalances` represents Aave `aToken` scaled units
- `requests[i].amount` represents confidential token units (`cToken`)
- These two values are in different token systems with different precision and scaling rules

Despite this, the protocol treats them as directly subtractable quantities, effectively mixing interest-accruing Aave internal accounting units with encrypted wrapped ERC20 representation units

This leads to incorrect balance updates and permanent accounting drift.

Root Cause

The scaled balance is not normalized using liquidity index before being subtracted by `requests[i].amount`.

Impact

collateral state diverges from actual Aave positions. Users may lose more collateral than intended due to incorrect subtraction.

Remediation

FIXED

H-07 - Missing Liquidation Handling Causes LTV Desync, Borrow Denial and Broken State Consistency

High

Description

The protocol does not implement any mechanism to handle Aave liquidations. Over time, Aave positions can get close to liquidation due to interest accrual on debt (variable debt growth).

Users can borrow any asset against a collateral. The aToken is held by the protocol. With time interest will accrue. The protocol has no way to enforce users to pay back. As a result when liquidation happens, the aToken and debt token held by the protocol will be burned. Since aToken and debt token balance changes after liquidation, that new balance will not reflect to all the user's scaled supplied and debt balances stored in storage.

This leads to a critical mismatch between:

- what the protocol believes is borrowed and supplied
- what it actually holds

Root Cause

No liquidation handling causing mismatch between stored scaled balance and actual balance

Impact

Some users won't be able to withdraw as those aTokens were burned during liquidation. And some users won't be able to repay because a part of debt tokens were burned and repaid during liquidation

Remediation

FIXED

H-08 - Gateway Callback Functions Are Missing onlyGateway Access Control

High

Description

The TFHE Gateway calls back into Diamond with a decrypted plaintext amount after each batch decryption. These callbacks are supposed to be restricted to the Gateway contract. None of them are. Any external caller can invoke them directly with an arbitrary amount, triggering real Aave operations on Diamond's account.

Diamond inherits `GatewayCaller`, which provides an `onlyGateway` modifier. But the callback functions live in separate facet contracts executed through Diamond's `fallback()` via `delegatecall`. The modifier is defined on Diamond and is never referenced in any facet or library. Diamond's fallback routes any matching selector to the registered facet with no caller check of its own.

Root Cause

`SupplyFacet.sol:13`, `WithdrawFacet.sol:13`, `BorrowFacet.sol:20`,
`RepayFacet.sol:19` - no modifier on any:

```
function callbackSupplyRequest(uint256 requestId, uint64 amount) external { ... }
function callbackWithdrawRequest(uint256 requestId, uint64 amount) external { ... }
function callbackBorrowRequest(uint256 requestId, uint64 amount) external { ... }
function callbackRepayRequest(uint256 requestId, uint64 amount) external { ... }
```

`Diamond.sol:36-62` - fallback makes no distinction on caller:

```
fallback() external payable {
    address facet = ds.selectorToFacetAndPosition[msg.sig].facetAddress;
    require(facet != address(0), "Diamond: Function does not exist");
    assembly {
        calldatacopy(0, 0, calldatasize())
        let result := delegatecall(gas(), facet, 0, calldatasize(), 0, 0)
        // ...
    }
}
```

Attack Path

Active request IDs are observable on-chain from `EventDecryption` events emitted by `GatewayContract` when a batch is submitted. An attacker monitors these and acts before the Gateway responds.

The most direct path is through the withdraw callback. `LibWithdrawRequest.sol:119-122`:

```
uint256 amountToWithdraw = amount *
    (10 ** (IERC20Metadata(asset).decimals() - ConfidentialERC20Wrapped(cToken).
        decimals()));
s.aavePool.withdraw(asset, amountToWithdraw, address(this));
```

1. A batch of withdraw requests reaches the threshold and a decryption request is sent to the Gateway.
2. Attacker sees the EventDecryption event and the request ID on-chain.
3. Attacker calls `Diamond.callbackWithdrawRequest(requestId, 1e6)` directly.
4. Fallback routes to `WithdrawFacet` via `delegatecall`. No caller check anywhere in the path.
5. $\text{amountToWithdraw} = 1\text{e}6 * 1\text{e}12 = 1\text{e}18$ for an 18-decimal asset like WETH. Diamond withdraws 1 WETH from Aave to itself.
6. Attacker repeats for other pending request IDs, draining all of Diamond's Aave supply.

The borrow callback is similarly exploitable: `callbackBorrowRequest` stores the attacker-supplied amount in `s.requestIdToAmount[requestId]`, which `finalizeBorrowRequests` then uses as the Aave borrow amount, creating unbacked debt on Diamond's account. Front-running the legitimate Gateway response also permanently bricks the affected batch, since the real callback will either double-execute or revert on corrupted state.

Impact

`callbackWithdrawRequest` is direct, immediate theft - a single call drains Diamond's entire Aave supply for any asset with a pending withdraw batch. `callbackBorrowRequest` creates unbacked Aave debt against Diamond. `callbackSupplyRequest` and `callbackRepayRequest` burn Diamond's cToken holdings with an attacker-chosen amount. Any pending batch on any operation type is vulnerable. The only precondition is a valid request ID, which is public.

Remediation

FIXED

M-01 - Unsafe IERC20.approve() Pattern Breaks Compatibility with USDT-Like Tokens

Medium

Location

- File: contracts/confidentiality-adapters/aave/libraries/LibWithdrawRequest.sol
- callbackWithdrawRequest(), L129
- File: contracts/confidentiality-adapters/aave/libraries/LibSupplyRequest.sol
- callbackSupplyRequest(), L125
- File: contracts/confidentiality-adapters/aave/libraries/LibBorrowRequest.sol
- _handleBorrowFinalize(), L179
- File: contracts/confidentiality-adapters/aave/libraries/LibRepayRequest.sol
- callbackRepayRequest(), L133; _repayAndGetDebtDelta(), L172

Root Cause

All five IERC20.approve() calls use the standard one-step pattern:

Vulnerable Code

```
// LibWithdrawRequest.sol -- L129
IERC20(asset).approve(cToken, amount);

// LibSupplyRequest.sol -- L125
IERC20(asset).approve(address(s.aavePool), unwrappedAmount);

// LibBorrowRequest.sol -- L179
IERC20(asset).approve(cToken, amount);

// LibRepayRequest.sol -- L133
IERC20(asset).approve(address(s.aavePool), unwrappedAmount);

// LibRepayRequest.sol -- L172
IERC20(asset).approve(address(s.aavePool), amount);
```

USDT (and some other tokens) implement a non-standard approve() that reverts if the current allowance is non-zero and the new approval is also non-zero. This is a well-documented behavior (Tether USD on Ethereum: require(!((_value != 0) && (allowed[msg.sender][_spender] != 0)))).

Scenario

1. A batch callbackSupplyRequest processes successfully but the subsequent aavePool.supply() or wrap() does not consume the entire allowance (e.g., Aave

returns early due to a cap, or the operation reverts and the allowance remains).

2. The next batch targeting the same asset calls `IERC20(asset).approve(spender, newAmount)`.

3. For USDT, this reverts because the remaining allowance from step 1 is non-zero.

4. All subsequent batches for this asset are permanently stuck – every new approve attempt reverts.

Impact

If any USDT-like token is used as an underlying asset in the Aave pool, and any approve is not fully consumed, all future callback operations for that asset will permanently revert. Since the protocol is designed as a generic Aave adapter supporting any ERC-20 asset, this is a realistic failure mode.

Recommendation

Use OpenZeppelin's `SafeERC20.forceApprove()` (or approve to 0 first):

```
import { SafeERC20 } from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
using SafeERC20 for IERC20;

IERC20(asset).forceApprove(spender, amount);
```

Remediation

FIXED

M-02 - Uninitialized matchedIndexes Slots Can DoS Mixed-Asset Supply Batches

Medium

Description

When `supplyRequest()` is triggered and `s.supplyRequests.length` reaches `REQUEST_THRESHOLD`, the function creates two fixed-size arrays and attempts to fill them with matched requests for the current asset:

```
LibAdapterStorage.SupplyRequestData[] memory requests = new LibAdapterStorage.  
SupplyRequestData[] (s.REQUEST_THRESHOLD);
```

```
uint256[] memory matchedIndexes = new uint256[] (s.REQUEST_THRESHOLD);  
uint256 count = 0; for (uint256 i = 0; i < s.supplyRequests.length; i++) {  
  LibAdapterStorage.SupplyRequestData memory srd = s.supplyRequests[i]; if  
  (srd.asset == asset) { requests[count] = srd; matchedIndexes[count] = i; count++;  
  if (count == s.REQUEST_THRESHOLD) break; } } If s.supplyRequests contains re-  
quests for different assets, count may never reach REQUEST_THRESHOLD for the  
current asset – leaving trailing slots in matchedIndexes uninitialized (defaulting  
to 0 in Solidity). The condition to call {_}processSupplyRequests is:
```

```
if (requests.length >= s.REQUEST_THRESHOLD) {
```

```
{_}processSupplyRequests(s, requests, matchedIndexes); }
```

 Since `requests.length` is always equal to `REQUEST_THRESHOLD` (initialized to that size), this check is always true – `{_}processSupplyRequests` is called even with partially filled arrays.

Root Cause

The deletion loop inside `{_}processSupplyRequests` iterates `matchedIndexes.length` times instead of `count` times:

```
for (uint256 i = matchedIndexes.length; i > 0; i--) {
```

```
  uint256 idx = matchedIndexes[i - 1]; s.supplyRequests[idx] =  
  s.supplyRequests[s.supplyRequests.length - 1]; s.supplyRequests.pop();  
}
```

 This means it processes uninitialized slots where `matchedIndexes[i] = 0` – using them as valid indexes into `s.supplyRequests`.

Proof of Concept

`REQUEST_THRESHOLD = 3` `s.supplyRequests = [ETH(0), USDC1(1), USDC2(2)]`

Third user supplies USDC -> s.supplyRequests.length == 3 == REQUEST_THRESHOLD -> batching branch triggered

Loop fills: matchedIndexes[0] = 1 // USDC1 at index 1 matchedIndexes[1] = 2 // USDC2 at index 2 matchedIndexes[2] = 0 // never written, defaults to 0

requests.length = 3 >= 3 -> _processSupplyRequests called

Deletion loop:

i=3: idx = matchedIndexes[2] = 0 (uninitialized slot) s.supplyRequests[0] = s.supplyRequests[2] // ETH wrongly overwritten pop() s.supplyRequests = [USDC2, USDC1], length = 2

i=2: idx = matchedIndexes[1] = 2 s.supplyRequests[2] = ... // index 2 no longer exists, length is 2 // dynamic storage arrays cannot be written beyond length - 1 -> OUT OF BOUNDS REVERT [REVERT] Transaction reverts – all state changes roll back.

Impact

Any time s.supplyRequests contains requests for mixed assets, batch processing always reverts

Recommended Mitigation

Pass count to _processSupplyRequests and loop that instead of matchedIndexes.length:

```
for (uint256 i = count; i > 0; i--) {
```

```
    uint256 idx = matchedIndexes[i - 1];    s.supplyRequests[idx] =  
    s.supplyRequests[s.supplyRequests.length - 1];    s.supplyRequests.pop();  
} This ensures only actually matched indexes are processed in the deletion loop,  
preventing both the wrong deletion and the out-of-bounds revert.
```

Remediation

FIXED

M-03 - Decimal Underflow When cToken Decimals Exceed Underlying Asset Decimals

Medium

Description

Inside `callbackSupplyRequest`, the protocol converts the decrypted cToken amount to the underlying asset's denomination:

```
// lines 120-121
uint256 unwrappedAmount = amount *
    (10 ** (IERC20Metadata(asset).decimals() - ConfidentialERC20Wrapped(cToken).
        decimals()));
```

Both `.decimals()` calls return `uint8`. The subtraction is performed in `uint8` arithmetic. Under Solidity 0.8+, if `cToken.decimals() > asset.decimals()`, this subtraction underflows and reverts with an arithmetic panic. There is no validation at the point of `setCTokenAddress` registration or here in the callback to enforce that `asset.decimals() >= cToken.decimals()`.

Root Cause

The subtraction between two `uint8` values is not guarded against underflow, and no invariant is enforced at token registration time to ensure the decimal relationship is valid.

Attack Path

This is a configuration failure mode. If a cToken is registered via `setCTokenAddress` with more decimals than its underlying asset (e.g., a wrapped token with 18 decimals paired with a 6-decimal USDC), every call to `callbackSupplyRequest` for that asset will revert permanently. Since this is the Gateway callback, it cannot be retried or re-ordered – the batch is stuck.

Impact

The entire supply flow for any asset with a misconfigured cToken decimal relationship is permanently bricked. All user cTokens deposited for that asset via `supplyRequest` are already transferred into the Diamond and cannot be unwrapped or returned. Funds are permanently locked.

Remediation

FIXED

M-04 - Lazy maxBorrowable Caching Uses Stale LTV After Aave Risk Parameters Change

Medium

Description

The adapter updates `userMaxBorrowablePerAsset` lazily. The value is refreshed only when the user passes through one of the local state-update flows in `LibSupplyRequest`, `LibBorrowRequest`, `LibWithdrawRequest`, or `LibRepayRequest`. Each of those paths calls `_setMaxBorrowables`, which reads the current reserve LTV from Aave at that moment and stores the computed result in adapter storage.

The problem is that the protocol later relies on this cached value during `LibBorrowRequest.borrowRequest` instead of revalidating borrowing power against Aave's current configuration. If Aave governance reduces the LTV of a reserve after the user's cached `maxBorrowable` was last written, the adapter continues to use the old higher value until the user happens to trigger another refreshing action.

That creates a stale-risk window where the borrow workflow validates a new borrow against outdated parameters even though the external protocol that ultimately holds the debt has already tightened its collateral requirements. Because Aave configuration is external to this adapter, the cache can become stale without any action from the affected user.

Impact

After Aave lowers an asset's LTV, users can still borrow against the old cached limit until their `maxBorrowable` is lazily refreshed. This allows borrowing beyond what the current Aave risk configuration should permit.

Root Cause

The protocol stores a derived risk-control value that depends on external Aave governance parameters, but it does not invalidate or recompute that value when those parameters change. `borrowRequest` trusts the cached `userMaxBorrowablePerAsset` value instead of deriving borrowing capacity from the current reserve configuration at the time of borrow validation.

Vulnerability Path

1. A user supplies collateral or otherwise triggers `_setMaxBorrowables`, causing

the adapter to cache a `userMaxBorrowablePerAsset` value based on the then-current Aave LTV.

2. Later, Aave governance lowers the LTV for the relevant reserve.

3. The user's cached `maxBorrowable` is not refreshed because no local adapter flow has updated it yet.

4. The user submits a borrow request, and `borrowRequest` validates the request against the stale cached `userMaxBorrowablePerAsset` value instead of the current Aave LTV.

5. The borrow batch finalizes and the adapter opens more debt than should be allowed under the updated Aave risk configuration.

Remediation

FIXED

M-05 - Borrow Finalization Rounds User Debt Down Below The Diamond's Actual Aave Liability

Medium

Description

Within the Aave confidentiality adapter, borrow requests are batched and then settled against Aave through the Diamond's single shared debt position. After `finalizeBorrowRequests()` executes the real Aave borrow, the adapter measures the Diamond's actual scaled-debt increase and tries to redistribute that increase back to users through a derived `multiplier`.

That redistribution rounds down twice:

- `LibBorrowRequest._handleBorrowFinalize()` computes `multiplier = difference / (amount / 1e6)`.
- `LibBorrowRequest._processStateUpdates()` then records each user's debt as `requests[i].amount * multiplier / 1e6`.

Because both divisions are integer truncations and no remainder is ever re-assigned, the sum of user debt increases can be smaller than the Diamond's real scaled-debt increase on Aave. In protocol context, that means the shared adapter account can owe more debt to Aave than the sum of what users are recorded as owing internally.

Impact

Users can be undercharged relative to the actual debt opened on the Diamond's Aave position.

The shortfall is socialized at the adapter level because Aave only sees the Diamond's aggregate debt, not the per-user confidential accounting. Over repeated borrow batches, these truncation losses can accumulate into protocol-borne bad debt and distort downstream accounting that relies on `scaledDebts[user][asset]`.

Root Cause

The borrow settlement logic converts a batch-level scaled-debt delta into per-user debt updates by flooring the batch `multiplier` and then flooring each user's share again, without assigning the discarded remainder to anyone.

Vulnerability Path

1. Let the Diamond's actual scaled-debt increase for a borrow batch be Δ .
2. Let the normalized batch amount be $T = \text{amount} / 1e6$, and let a user's normalized request be $t_i = \text{requests}[i].\text{amount} / 1e6$.
3. The code computes $\text{multiplier} = \text{floor}(\Delta / T)$.
4. Each user is then assigned $\text{recorded}_i = \text{floor}(t_i * m)$.
5. Therefore $\text{sum}(\text{recorded}_i) \leq T * \text{floor}(\Delta / T) < \Delta$ whenever Δ is not perfectly divisible by T .
6. The missing remainder is never assigned, so user debt is rounded down below the Diamond's real borrow-side liability.

Recommendation

Round debt accounting in favor of the protocol when converting the batch delta into user debt.

For example, use ceil-style rounding for the batch multiplier and/or the per-user debt allocation, or explicitly assign the final remainder to one user or to a protocol-owned dust bucket, so the total recorded debt is never lower than the Diamond's actual Aave debt.

Remediation

FIXED

L-01 - onUnwrap - Unbounded Backward Loop Scales as $O(\text{requestId})$, Leading to Out-of-Gas

Low

Location

- Contract: Diamond
- File: contracts/confidentiality-adapters/aave/Diamond.sol – onUnwrap(), L73-L78

Root Cause

The loop for (uint256 i = requestId - 1; i > 0; i-) has no upper bound on iteration count. It decrements from requestId - 1 down to 1, performing a cold SLOAD (requestIdToRequestData[i]) on each iteration.

Vulnerable Code

```
// Diamond.sol -- L73-L78
for (uint256 i = requestId - 1; i > 0; i--) {
    LibAdapterStorage.RequestType rt = s.requestIdToRequestData[i].requestType;
    if (rt == LibAdapterStorage.RequestType.SUPPLY || rt == LibAdapterStorage.
        RequestType.REPAY) {
        s.requestIdToUnwrapRequestId[i] = requestId;
        break;
    }
}
```

Impact

Each cold SLOAD costs 2,100 gas. If no matching entry is found (or with H-01 fixed so that uninitialized entries no longer match), the loop runs up to requestId - 1 iterations. At requestId ~ 14,286, the gas cost reaches ~30M, which is the Ethereum block gas limit. Even with warm SLOADs (100 gas), the limit is breached at requestId ~ 300,000. Once this threshold is crossed, onUnwrap permanently reverts for all subsequent request IDs, and no supply or repay batch can ever be finalized again.

Eliminate the backward-scanning loop. Instead, maintain a storage variable lastSupplyOrRepayRequestId that is updated each time a new SUPPLY or REPAY request is created. onUnwrap can then directly write s.requestIdToUnwrapRequestId[lastSupplyOrRepayRequestId] = requestId in $O(1)$.

Remediation

FIXED

L-02 - getSuppliedBalance Returns Scaled Balance Instead of Interest-Accrued Balance

Low

Description

The `GetterFacet::getSuppliedBalance` function is intended to return a user's actual supplied balance, including accrued interest. However, it currently returns only the stored scaled balance without applying the asset's supply index. As a result, the returned value significantly underreports the user's true balance over time.

Root Cause

The function directly returns `s.scaledBalances[user][asset]` without multiplying it by the corresponding supply index (or liquidity index). In Aave, user balances are stored in scaled form and must be multiplied by the current index to derive the real balance.

Impact

Users and integrators receive incorrect balance data, leading to underestimation of holdings.

Remediation

FIXED

L-03 - Encrypted Borrowed Balance Returned Without ACL Permission Prevents Users From Decrypting Debt Value

Low

Description

In `GetterFacet::getBorrowedBalance`, the function correctly computes the user's real debt (including interest accrual) using Aave's normalized variable debt index:

```
euint256 scaledResult = TFHE.div(  
    TFHE.mul(TFHE.asEuint256(scaledDebt), reserveNormalizedDebt),  
    1e27  
);
```

However, the function returns the result as an encrypted value:

```
return TFHE.asEuint64(scaledResult);
```

No ACL (`TFHE.allow`) permission is granted to the user, meaning the returned value exists only in encrypted form and the user cannot decrypt or view their own debt position. As a result, the function becomes unusable for end users despite being a "getter".

Root Cause

The user is not granted permission in ACL to decrypt the newly calculated debt including interest. As a result, users will get encrypted amount but won't be able to do anything

Impact

Users cannot view their own debt due to lack of permission in ACL

Remediation

FIXED

I-01 - Division by Zero in Multiplier Calculation When Total Batch Amount Is Less Than 1e6

Info

Location

- File: `contracts/confidentiality-adapters/aave/libraries/LibSupplyRequest.sol`
– `finalizeSupplyRequests()`, L148
- File: `contracts/confidentiality-adapters/aave/libraries/LibBorrowRequest.sol`
– `_handleBorrowFinalize()`, L180

Root Cause

The multiplier is computed as:

Vulnerable Code

```
// LibSupplyRequest.sol -- L148
uint256 multiplier = difference / (amount / (10 ** 6));
```

The inner expression `amount / (10 ** 6)` performs integer division. When `amount` < `1_000_000` (i.e. the decrypted total batch sum represents less than 1 full token in 6-decimal representation), this evaluates to 0. The outer division `difference / 0` triggers a Solidity 0.8.x `Panic(0x12)` – arithmetic division by zero.

Scenario

`REQUEST_THRESHOLD` is set to 3. Three users each submit a supply request for 0.3 tokens (300,000 in 6-decimal). The Gateway decrypts the sum as 900_000. The `finalize` function computes `900_000 / 1_000_000 = 0`, then divides the Aave scaled-balance difference by zero. The transaction reverts, and the batch – along with all enclosed user funds – is permanently locked. The Gateway has already processed the decryption, so re-submitting the same request is not possible.

Impact

Any batch whose total decrypted amount is below 1 token (in 6-decimal scale) causes an irrecoverable revert in the finalization step. User funds within the batch cannot be recovered as there is no alternative code path to handle this state, and the request data is consumed by the Gateway callback.

Recommendation

Reorder the arithmetic to multiply before dividing:

```
uint256 multiplier = (difference * (10 ** 6)) / amount;
```

Additionally, add a minimum-amount check after decryption or in the finalization path to ensure `amount > 0`.

Remediation

ACKNOWLEDGED

I-02 - Expired Gateway Deadlines Can Lock Batched Supply And Repay Funds Inside The Adapter

Info

Description

The Aave adapter's `supplyRequest()` and `repayRequest()` flows pull confidential wrapped tokens from the user into the Diamond before the asynchronous decryption step completes:

- `LibSupplyRequest.supplyRequest()` transfers the user's `cToken` to the adapter, stores the request, and once the threshold is reached calls `Gateway.requestDecryption(..., block.timestamp + 100, ...)`.
- `LibRepayRequest.repayRequest()` does the same for repay batches, using `block.timestamp + 500`.

If the relayer/KMS fulfills either request after the configured expiry, `GatewayContract.fulfillRequest()` sets `isFulfilled[requestID] = true` and then reverts with `Too late`, so the callback is never executed and the original gateway request cannot be retried through the intended path since those requests are deleted once processed.

That breaks the rest of the adapter pipeline:

- `callbackSupplyRequest()` / `callbackRepayRequest()` never run, so `ConfidentialERC20Wrapped(cToken).unwrap(...)` is never called.
- `Diamond.onUnwrap()` never receives the `unwrap` callback, so `requestIdToUnwrapRequestId` and `requestIdToAmount` are never populated for the affected supply/repay batch.
- `finalizeSupplyRequests()` later reads `requestIdToAmount[unwrapRequestId] == 0` and reverts with `AmountIsZero()`.
- `finalizeRepayRequests()` reverts with `NoUnwrapRequestIdFound()` or `AmountIsZero()`.

There is also no explicit cancel, retry, refund, or rescue function in the Diamond facets that returns these already-collected `cTokens` to users or requeues the batch. As a result, the intended protocol flow deadlocks after expiry.

Impact

Users can lose access to already-transferred `cTokens` indefinitely if a supply or repay batch misses the gateway deadline.

For supply, the deposited confidential tokens have already been moved from the user to the adapter, but the batch never reaches Aave and the user's internal supplied balance is never updated.

For repay, the user's confidential tokens are likewise already pulled into the adapter, but the debt repayment never executes and the user's debt accounting remains unchanged.

Because the gateway marks the expired request as fulfilled before reverting, the original decryption request is effectively dead. With no explicit refund or retry path in the adapter, the batch remains stuck and the user's funds stay trapped in the Diamond adapter.

Root Cause

The adapter transfers user funds into protocol custody before the asynchronous gateway callback is finalized, while also relying on short hardcoded expiry windows for that callback.

When the callback is missed, the adapter has no state transition for expired requests:

- no retry of the original decryption request,
- no refund of the already-collected `cTokens`,
- no batch cancellation,
- and no alternate sanctioned settlement path.

Remediation

ACKNOWLEDGED

Disclaimers

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource, and expertise-bound effort in which we aim to identify as many issues as possible.

About Kann Audits

Kann Audits is a Web3 security audit company providing manual smart contract reviews, AI-assisted security analysis, and security guidance for blockchain protocols.

To learn more, visit <https://kannaudits.com>

To view our audit portfolio, visit <https://github.com/KannAudits>

To book an audit, message <https://t.me/KannAudits>